

# YAI Agent Core —— 进程内嵌入式自适应 Agent 内核

2026 上海开源软件应用创新大赛 · 开源 AI 工具赛道参赛作品介绍 作者：Gi-Tuu（单人，队名 YAI） | 仓库：<https://github.com/Gi-Tuu/yai-agent-core> 在线实例：<https://yai-agent-core.onrender.com> | 许可证：MIT

## 一、项目背景：软件"AI 化"的成本被严重低估

大模型能力越来越强，但一个已经存在的软件（笔记应用、CRM、陪伴 App、内部系统）要获得 Agent 能力，今天的主流路径仍然是：选一个 Agent 框架，自己接模型、定义工具、写 Agent Loop、写规划器、写记忆、做权限、做可观测、做部署运维。市面上的方案大致分三类：

1. **托管平台**（Dify、Coze 等）：可视化编排、开箱即用，但应用必须运行在平台里或以平台为服务端，数据、UI、发布节奏都在平台侧，存量软件无法"内嵌"。
2. **拼装框架**（LangChain/LangGraph、CrewAI、AutoGen 等）：提供积木与编排范式，但"谁是 Agent、有哪些工具、流程怎么转"仍需开发者逐一定义与装配，框架本身不感知宿主软件已有什么能力。
3. **自建 Agent**：可控但重复造轮子，每个软件都要重写一遍循环、工具总线和记忆。

**YAI Agent Core** 提出另一种形态：进程内嵌入式的 Agent 内核（对标数据库世界里的 SQLite，而不是 ORM 或云数据库控制台）。宿主软件只做一件事——声明"我有什么能力"（普通 Python 函数加类型注解与一行 docstring），内核在进程内自动完成能力发现、意图理解、策略选择、工具调用、记忆与可审计交付，**宿主不写一行 Agent Loop / Planner / 工具选择代码**。

```
# 宿主侧全部接入成本（examples/host_a_notes/run.py 同构）
from yai_core import AgentCore
core = AgentCore.auto(capabilities, model) # capabilities 是普通函数
result = await core.run("帮我统计一下笔记数量")
# 内核自适应选择 direct/react/plan/clarify
```

这一形态直接服务于"应用 AI 化"的真实成本：能力长在应用进程里，不引入服务端、不锁定模型厂商、不抽成、数据不出宿主。

## 二、技术架构

### 2.1 分层结构

宿主软件（host\_a / host\_b / host\_c / host\_e / 任意 Python 软件 / AMBRACE）

| 声明 capabilities（普通函数）+ 选择 model/channel/memory/policy



YAI Agent Core（进程内库，内核本体零第三方硬依赖，dependencies = []）

| 模块                       | 职责                                                    |
|--------------------------|-------------------------------------------------------|
| discovery/introspect     | 能力自发现：函数 type hints + docstring<br>→ JSON Schema      |
| kernel/router            | 自适应路由 direct/react/plan/clarify（规则兜底 + LLM 增强）        |
| kernel/context           | 上下文组装与字符预算压缩（轮边界对齐）                                   |
| kernel/loop              | Agent Loop: ReAct 循环 / 先规划后执行 / 澄清 / 收尾               |
| tools/                   | ToolRegistry + ToolExecutor（Schema 校验 → 权限 → 执行）      |
| discovery/catalog        | 按需能力目录：ToolDiscovery 的最小进程内实现（关键词匹配）                  |
| policy/allowlist         | 权限收口（读放行 / 写授权）                                       |
| memory/                  | 内存 + SQLite 持久化（opt-in）+ 条数/TTL 保留策略                  |
| spi/                     | model / channel / memory / policy / discovery 五个可替换契约 |
| integrations/mcp         | MCP Client（Streamable HTTP / stdio / 进程内直连）           |
| integrations/openapi     | OpenAPI 3 发现 → 工具自动桥接（\$ref 内联、只读模式、鉴权）               |
| batteries/fastapi_server | /health、/v1/tools、/v1/agent/run + 限流（可选）              |

## 2.2 核心机制

(1) 能力自发现。宿主的普通函数经内省自动生成 JSON Schema 工具规格；外部 MCP Server 工具与 OpenAPI 服务操作被转换成同构的 ToolSpec 注册进同一张工具表，执行侧对工具来源零感知。目前工具来源有三类：native 函数、MCP Server（远程/本地）、OpenAPI REST 服务。

(2) 自适应路由（四策略）。每个任务进入内核后先做一次路由：

- direct: 无需工具，模型直接回答；
- react: ReAct 循环，边推理边调工具；
- plan: 多步任务先由模型产出有序计划（发 plan\_created 事件），再逐步执行；
- clarify: 意图不清时向用户反问，带轮数上限、上下文累积与空回答收尾，不会无限追问。

路由有两条路径：LLM 分类输出 {strategy, reason, tier, missing\_capability}，模型不可用、超时或输出非法时自动回退到确定性规则路由；每次决策都在事件流里标明 source=llm|rules，可审计、可复现。llm\_router="auto" 时按模型后端能力标记自动开关（真实模型开、离线脚本模型关）。

(3) 能力缺口的最小闭环：感知 → 发现 → 注册 → 本轮可用。当 LLM 判断现有工具不足以完成任务时，内核先发出 capability\_missing 事件（含缺口描述与可用工具清单）；若宿主注入了 ToolDiscovery 发现源，内核随即向它要候选、统一去重后注册进工具表、发出 tool\_discovered 事件，并在同一轮把新工具交给模型调用——不需要等下一次任务。内置的 StaticCatalog 是进程内零依赖实现（候选必须显式登记关键词才可能被发现），离线可复现；同一契约未来可挂 MCP 目录、OpenAPI 服务目录或插件市场。发现源异常只发错误事件、不中断主流程；更关键的是“能被发现不等于被授权执行”，新工具执行时仍走统一权限闸。不配置发现源时只发缺口事件、行为与旧版完全一致。

**(4) 工具总线与权限。** 工具执行统一经过 `JSON Schema` 参数校验、`PermissionPolicy` 授权、`sync/async` 执行与结果回灌；宿主可提供自己的权限策略，例如 `host_e` 中 8 个读工具自动放行、4 个写工具执行前人工授权。

**(5) 事件流唯一出口。** 策略选择、计划生成、工具调用与结果、权限确认、能力缺口、错误与完成，全部经 `AgentCore.astream` 以 `AgentEvent(type, data)` 流式发出，不存在静默决策。这让 UI（终端/网页/未来的 `Flutter`）只需要消费同一条事件流。

**(6) SPI 五契约与零硬依赖。** 模型、通道、记忆、权限、发现五个插槽均为 `Protocol` 契约，内核只依赖抽象；`pyproject.toml` 的 `dependencies` 为空，`openai / fastapi / mcp / httpx` 全部是可选 `extra` 并在模块内懒加载（有 `AST` 红线测试守护），内核因此可以进入装不下框架全家桶的受限环境。

**(7) 长期运行可信度。** 记忆支持 `SQLite` 持久化（`opt-in`）与条数上限/TTL 保留策略（按对话轮边界裁剪，不拆散工具调用对）；在线 `API` 提供每 `IP` 滑动窗口限流（超限 429 + `Retry-After`）与 `/health` 实时 `commit` 端点。

### 三、应用场景

内核的价值通过"同一内核换宿主即换岗位"的示例体系验证，仓库内置 6 个形态各异的宿主：

| 宿主            | 形态                 | 证明的事                               |
|---------------|--------------------|------------------------------------|
| host_a 笔记     | 办公提效               | 3 个本地函数即得"统计/检索/整理"数字员工            |
| host_b 销售数据   | 数据应用               | 同一内核零修改适配另一行业                      |
| host_c AI 陪伴  | 陪伴助手（AMBRACE 回流预演） | 档案/回忆/记忆/心情/关怀 5 工具，自适应换岗位         |
| host_d MCP    | 外部工具               | 接入公共 MCP Server（DeepWiki），远程工具真实调用 |
| host_e 销售 CRM | 完整软件               | 零 AI 依赖可独立运行的菜单软件，同一业务对象零改造嵌入内核    |
| host_f 能力发现   | 按需成长               | 离线确定性演示"缺口→发现→注册→本轮可用"，无需 Key      |

**host\_e 是"存量软件零改造嵌入"的关键证据：** `crm_app.py` 是一个不导入任何 `yai_core` 的完整 CRM（12 个业务方法，8 读 4 写，JSON 持久化，菜单 CLI 与网页界面）；嵌入时把同一个 `SalesCrm` 对象交给 `AgentCore.auto`，新增的只是一个约 30 行的入口文件——读操作自动执行，写操作弹授权，一句"查超期客户 → 写跟进 → 建待办 → 出日报"由内核自动规划并执行，结果实时反映在原生界面。它同时提供终端与纯标准库实现的网页工作台（`http.server` + SSE，零第三方依赖），打开首先是软件本来的样子，AI 是可收起的右侧抽屉。

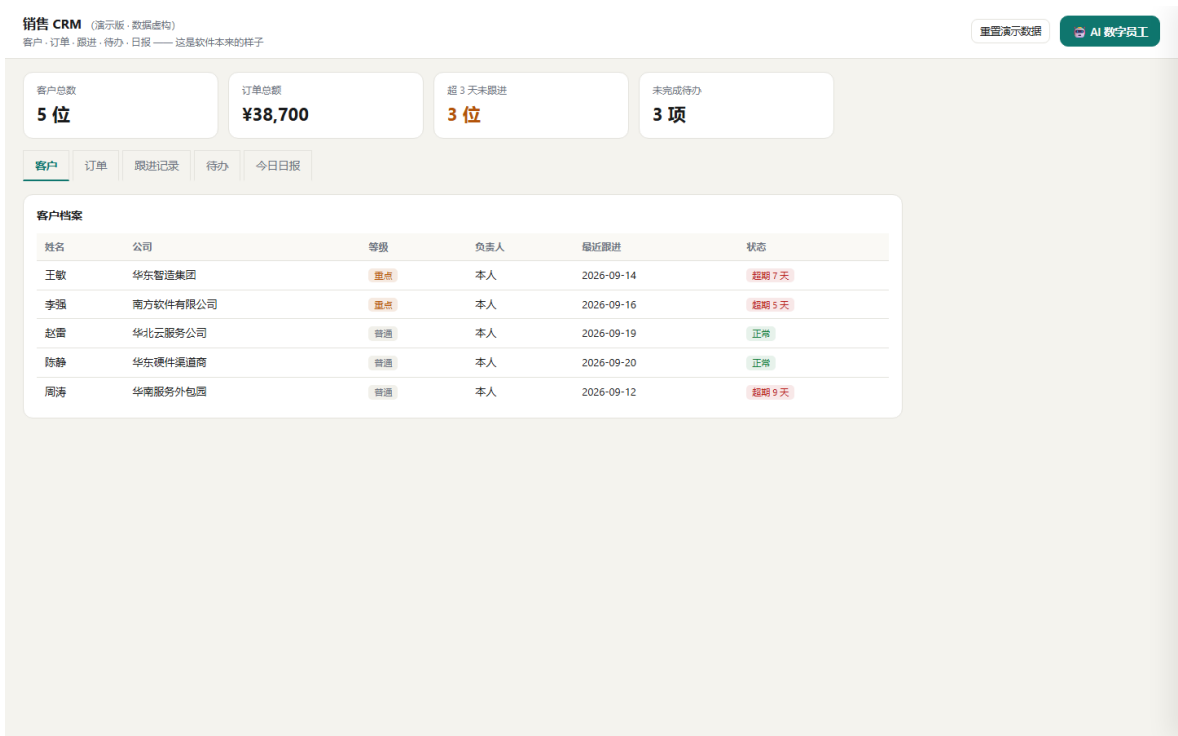


图 1：销售 CRM 原生界面——不依赖 Core 也完整可用（客户 / 订单 / 跟进 / 待办 / 日报五个视图，数据均为虚构）

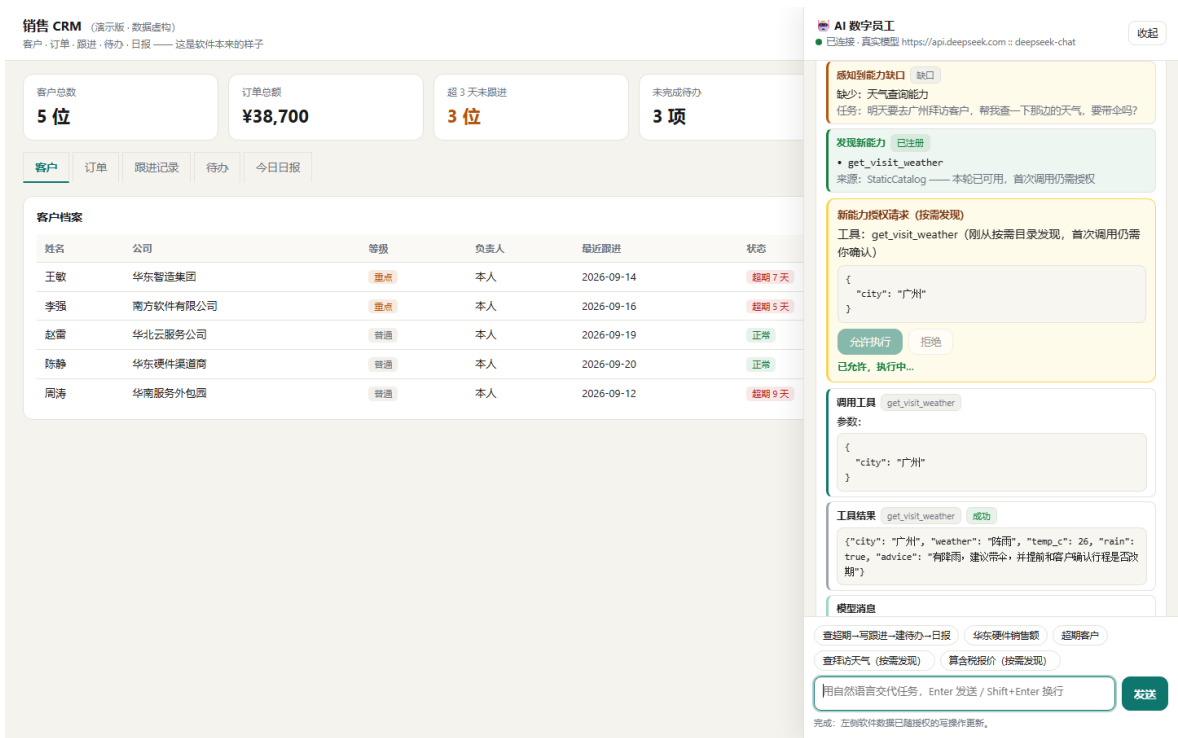


图 2：同一 CRM 嵌入 Core 后，右侧 AI 抽屉感知能力缺口（琥珀）→ 按需发现天气工具（绿）→ 新能力授权（黄）→ 调用，左侧原生 CRM 同框联动

**在线可验收：**实例 <https://yai-agent-core.onrender.com> 免登录提供 `GET /health`（实时返回 `commit`）、`GET /v1/tools`（本地工具 + 远程 `MCP` 工具清单）、`POST /v1/agent/run`（请求体 `{"task": "..."}` ，返回事件流与最终结果），评审可直接 `curl` 复验。

**可复现**（与 `CI` 完全一致，全程不需要 `API Key`）：

```
git clone https://github.com/Gi-Tuu/yai-agent-core.git && cd yai-agent-core
uv sync --extra dev --extra llm --extra server --extra mcp --extra openapi
uv run pytest # 219 项离线测试全绿（不依赖网络/Key）
uv run ruff check src tests examples scripts
uv run python scripts/smoke_test.py # 一个内核冒烟跑通三个宿主
docker compose up --build # 容器化（容器内 8000）
```



## 四、创新点

1. **范式反转: `host-declares-capability / core-adapts`**。 现有框架止步于"提供积木", 开发者仍需定义并装配 `Agent`; YAI 把内核做成可以被已有软件直接 `import` 的"AI 引擎", 宿主声明能力、内核自适应完成其余全部。对照上海赛评审"技术创新"维度关注的"是否解决已有方案未解决的问题、是否简单组合", 本项目的差异不在堆功能, 而在交付形态 (详见 `docs/comparison.md` 与五类方案的逐项对比)。
2. **自适应决策可审计、可确定性退化**。 路由、规划、工具选择、能力发现全部事件化; LLM 不可用时规则路径保证离线、零成本、可测试的下限——219 项测试全部不依赖网络与密钥, 这在内核类项目中并不常见。
3. **能力缺口能被"补上", 而不只是被"喊出"**。 多数框架的工具集合是写死的; YAI 让内核在感知到 `capability_missing` 后, 经 `ToolDiscovery` 发现源把缺失能力在本轮注册并使用 (`capability_missing` → `tool_discovered` → `tool_call` → `tool_result`), 发现不等于授权、异常不致命。这是"发现工具/创造工具"演进路线上可运行的第一步, 而非停留在事件定义。
4. **内核零硬依赖 + SPI 五契约**。 模型可换 (OpenAI 兼容, 当前演示用 DeepSeek)、记忆可换、通道可换、权限可换、发现源可换; 依赖合规是开源治理的天然优势。
5. **双协议工具发现**。 同一工具总线统一 `native` 函数、MCP Server、OpenAPI 服务三类来源, 宿主的工具生态可以随协议自然扩展。

## 五、开源治理与长期发展

- **协议与依赖合规**：MIT 许可证；内核本体 `dependencies = []`，第三方依赖全部为可选 `extra` 且懒加载，无密钥、无数据上报，`.env` 永不入库。
- **工程质量**：219 项离线测试（假模型 / `httpx MockTransport` / 注入时钟，不依赖网络与 `Key`）、`Ruff`、`GitHub Actions` 四矩阵（`Ubuntu 3.11/3.12/3.13 + Windows 3.13`）、`Conventional Commits`。
- **社区资产**：`CONTRIBUTING.md`（含新人 30 分钟跑通）、`Issue/PR` 模板、`ROADMAP.md`（明确每版范围与“不做什么”）、`docs/comparison.md` 诚实对比、16 篇源码逐行讲义（`docs/walkthrough/`，面向学习者的完整文档）。
- **长期路线**：`ToolDiscovery` 最小闭环已落地（进程内静态目录）→ 下一版本两层工具目录与每轮反思（支撑上百工具不爆上下文）→ 动态发现源（`MCP` 目录 / `OpenAPI` 服务目录 / 插件市场）→ `v1.0` 回流开源 AI 陪伴项目 `AMBRACE`（<https://github.com/Gi-Tuu/AMBRACE>），以真实 To C 软件验证内嵌路径，形成“比赛验证—开源沉淀—产品内嵌”的闭环。
- **明确不做**（防止内核膨胀、边界清晰）：`MCP Server`、`Multi-Agent`、向量记忆、内置 `UI`、`coding agent`——这些属于宿主层或独立插件，已在 `ROADMAP` 中公开说明。

## 六、诚实边界

- 在线实例为免费 PaaS (Render)，无流量时休眠，冷启动约 30–90 秒；免费层临时盘重建即清空 SQLite（演示数据本为虚构）。
- 限流不防御伪造 X-Forwarded-For，仅防误刷，代码注释如实声明；鉴权/API Key 在路线图中。
- 工具发现目前只落地了**进程内静态目录**这一最小闭环（显式关键词匹配、离线可测）；运行时从 MCP 目录 / OpenAPI 服务目录 / 插件市场做语义发现、以及工具组合/生成 (v1.x) 尚未实现，不夸大自动化程度。
- 所有演示数据均为虚构，无真实个人数据。